

LABORATORY

LAB 08

Temperature Measurement with NTC

Reading Temperature with the NTC Device API in EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	NTC Thermistor	2
2.2	Voltage Divider and Analog Signal	3
2.3	From Analog Signal to Temperature	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Pin Mapping	4
3.3	Connecting the NTC Module	4
4	EduFramework API	5
4.1	<code>NTC_Init()</code>	5
4.2	<code>NTC_ReadCelsius()</code>	5
5	Lab Exercise	6
5.1	Requirements	6
5.2	Program	6
5.3	Verification	7
6	Extension	7
6.1	Observing Voltage with <code>NTC_ReadMilliVolts()</code>	7
6.2	Observing Resistance with <code>NTC_ReadResistance()</code>	8
6.3	Additional NTC APIs	8
7	References	8

1 Introduction

1.1 Lab Overview

Temperature is a physical quantity commonly monitored in measurement and control systems. A thermistor is a resistive element whose resistance changes with temperature; for an NTC (Negative Temperature Coefficient) thermistor, resistance decreases as temperature increases [1] .

In previous labs, analog signals were read as ADC values or used directly to control an output. This lab moves to the Device layer of EduFramework: the electrical signal from the NTC module is processed through the ADC and then converted into resistance and temperature so that the application can work directly with the physical quantity in degrees Celsius [2] [6] .

The lab exercise uses `NTC_ReadCelsius()` to read temperature from the NTC module and display the result on the Serial Monitor. Intermediate values such as voltage and resistance are reserved for the Extension section.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the basic operating principle of an NTC thermistor and the relationship between temperature and resistance.
2. Explain how a voltage divider converts the resistance variation of an NTC thermistor into an analog voltage that can be measured by an ADC.
3. Describe the conversion path from the analog signal of the NTC module to a temperature value in EduFramework.
4. Use `NTC_Init()` and `NTC_ReadCelsius()` to read temperature in degrees Celsius.
5. Build and verify a temperature monitoring application using the Serial Monitor.

2 Background

2.1 NTC Thermistor

A thermistor is a temperature-dependent resistor commonly used as a temperature-sensing element. The two basic categories are NTC and PTC. With an NTC thermistor, resistance decreases as temperature increases; with a PTC thermistor, resistance increases as temperature increases. The resistance-temperature characteristic of a thermistor is nonlinear [1] .

In temperature measurement, the change in NTC resistance is an intermediate quantity. A microcontroller does not directly measure resistance with an ADC; the resistance variation must first be converted into a measurable voltage [2] .

Note

An NTC thermistor is a temperature-sensitive element with a nonlinear characteristic. Its resistance cannot be treated as temperature through a simple linear relationship over the entire measurement range [1] .

2.2 Voltage Divider and Analog Signal

A common method for measuring a thermistor with a microcontroller is to connect the NTC thermistor and a fixed resistor as a voltage divider. As the NTC resistance changes, the voltage at the midpoint also changes and can be applied to an ADC input [2] [3].

For the configuration used by the EduFramework NTC Device:

VREF → R_FIXED → ADC(A0) → NTC → GND

the voltage at the ADC input is determined by the voltage-divider relationship:

$$V_{\text{ADC}} = V_{\text{REF}} \times \left(\frac{R_{\text{NTC}}}{R_{\text{FIXED}} + R_{\text{NTC}}} \right)$$

From the measured voltage, the NTC resistance can be derived as:

$$R_{\text{NTC}} = R_{\text{FIXED}} \times \left(\frac{V_{\text{ADC}}}{V_{\text{REF}} - V_{\text{ADC}}} \right)$$

This relationship corresponds to the way `NTC_ReadResistance()` converts the measured voltage into resistance in the current EduFramework implementation [2] [6].

The S32K144 integrates an ADC to convert an analog voltage into digital data. EduFramework uses its Analog API layer to perform ADC measurements for the NTC Device [4] [6].

2.3 From Analog Signal to Temperature

The temperature measurement path in this lab can be represented as follows:

Temperature → NTC resistance → A0 voltage → ADC → Resistance → Temperature

The ADC measurement and resistance calculation steps are encapsulated by the Device API. Therefore, the application does not need to read the ADC and perform the voltage-divider calculation manually before obtaining a temperature value [6].

In the current implementation, `NTC_ReadCelsius()` calls `NTC_ReadResistance()` to obtain the resistance and then uses a lookup table for a 10 kΩ B3950 NTC thermistor with linear interpolation between adjacent points to determine the temperature. The lookup table covers temperature points from -20 °C to 80 °C; when the resistance lies outside the two ends of the table, the implementation returns the corresponding boundary temperature [6].

The MIT material presents the NTC characteristic as resistance-versus-temperature data, showing that different temperature regions correspond to different resistance ranges [1]. EduFramework currently applies discrete data based on this principle in its internal lookup table and interpolates between adjacent points [6].

3 Hardware Setup

3.1 Required Hardware

The lab uses the MaaZEDU Development Board together with an NTC module that provides an analog A0 output. MaaZEDU uses the S32K144 microcontroller; the module's A0 signal is connected to an ADC Logical Pin in EduFramework, and the resulting temperature is observed through the Serial Monitor [5] [6].

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
NTC temperature sensor module	NTC thermistor module with an analog AO output for temperature measurement.
Jumper wires	Connect power, GND, and the analog signal between the NTC module and MaaZEDU.
USB Cable	Connect the board to the computer for power, programming, and Serial Monitor communication.

Table 8.3.1. Hardware used in the lab exercise

3.2 Pin Mapping

The NTC Device API accepts an analog Logical Pin as its input parameter. This lab uses `ADC0_SE13` to receive the `A0` signal from the NTC module [6].

Component	Logical Pin	MCU Pin	Function
NTC AO	ADC0_SE13	PTC15	Analog Input / ADC0 Channel 13

Table 8.3.2. NTC signal mapping used in the lab exercise

3.3 Connecting the NTC Module

In the hardware configuration verified for this lab exercise, the NTC module is powered from 3.3V, shares GND with MaaZEDU, and provides its AO signal to ADC0_SE13. The module's DO pin is not used in the main exercise. The NTC Device analog measurement path supports ADC Logical Pins such as ADC0_SE12 and ADC0_SE13 [5] [6].

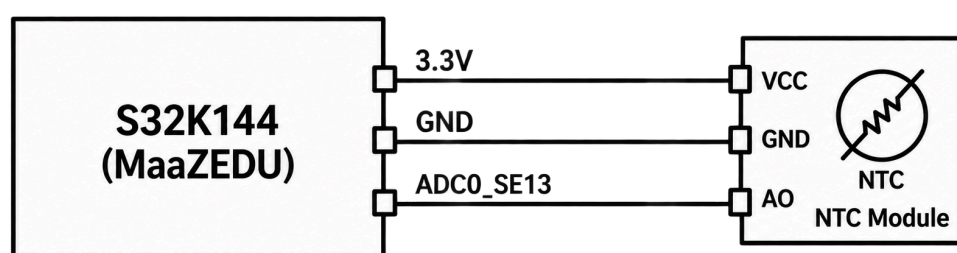


Figure 8.3.1. NTC module connection to the MaaZEDU Development Board

Note

The main exercise uses only the analog AO output. The digital DO output available on some NTC modules can be used for threshold detection and is introduced in the Extension section [6].

4 EduFramework API

The main exercise requires only two NTC Device APIs: `NTC_Init()` and `NTC_ReadCelsius()`. Voltage and resistance measurements, together with additional functions, are reserved for the Extension section [6].

4.1 NTC_Init()

`NTC_Init()` initializes the default configuration of the NTC Device.

NTC_Init

Initialize the default configuration of the NTC Device before performing measurements.

Syntax

```
NTC_Init();
```

Return value

No return value.

Example:

```
NTC_Init();
```

4.2 NTC_ReadCelsius()

`NTC_ReadCelsius()` reads the NTC analog signal, calculates the resistance, and returns the converted temperature in degrees Celsius. The API accepts the analog Logical Pin connected to the module's `A0` pin [6].

NTC_ReadCelsius

Read the NTC module through an Analog Input and return the temperature in degrees Celsius.

Syntax

```
NTC_ReadCelsius(pin);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	Analog Logical Pin	Analog pin connected to the <code>A0</code> output of the NTC module, for example <code>ADC0_SE13</code> .

Return value

Temperature in degrees Celsius when the measurement succeeds; `-273.15F` when the measurement or conversion fails.

Example:

```
float temperature = NTC_ReadCelsius(ADC0_SE13);
```

5 Lab Exercise

5.1 Requirements

Build a program that reads temperature from the NTC module through `ADC0_SE13` and displays the result on the Serial Monitor. The program initializes `Serial1` at a baud rate of `9600`, initializes the NTC Device once, and updates the temperature value every `1000 ms`.

After the program operates stably, change the temperature around the thermistor by gently touching it or placing the sensor near a safe heat source to observe the trend in the measured result. A fixed temperature value is not required; the objective is to verify that the measurement responds appropriately when the thermal condition changes.

5.2 Program

In `src/main.c`, implement the program that has been verified on hardware as follows:

```
#include "Arduino.h"
#include "ntc.h"

int main(void)
{
    float temperature = 0.0F;

    setup();

    Serial1_begin(9600U);
    NTC_Init();

    Serial1_println("NTC Temperature Monitor");

    while (1)
    {
        temperature = NTC_ReadCelsius(ADC0_SE13);

        Serial1_print("Temperature: ");
        Serial1_printFloat(temperature);
        Serial1_println(" C");

        delay(1000U);
    }

    return 0;
}
```

Listing 8.5.1. Program for reading temperature from the NTC module

`setup()` initializes the underlying components of EduFramework. `Serial1_begin()` initializes the Serial Monitor channel using the configuration introduced in Lab 03, while `NTC_Init()` initializes the default configuration of the NTC Device [6].

In the main loop, `NTC_ReadCelsius(ADC0_SE13)` performs the required NTC Device measurement path and returns the temperature in degrees Celsius. The result is printed to the Serial Monitor, and `delay(1000U)` then creates a one-second interval between updates.

The application processing flow can be summarized as:

Temperature → NTC module → `ADC0_SE13` → `NTC_ReadCelsius()` → `Serial1` → Serial Monitor [6].

5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then open the Serial Monitor at a baud rate of `9600`. Observe the temperature value while the sensor is in a stable environment, then safely warm the thermistor and verify the trend in the displayed value.

The Serial Monitor output has the following form:

```
NTC Temperature Monitor
Temperature: 28.91 C
Temperature: 29.02 C
Temperature: 29.14 C
Temperature: 29.27 C
```

EXPECTED RESULT

The Serial Monitor updates the temperature approximately every `1000 ms`. Under stable thermal conditions, consecutive values fluctuate within a similar range. When the thermistor is warmed, the displayed temperature tends to increase; when the heat source is removed and the sensor gradually cools, the value tends to return toward the ambient temperature. This response is consistent with the NTC principle and the conversion path implemented in the NTC Device [1] [6].

6 Extension

The main exercise uses only the final temperature value to keep the program concise. The NTC Device also provides APIs for observing intermediate measurement quantities or using other operating modes of the module [6].

6.1 Observing Voltage with `NTC_ReadMilliVolts()`

`NTC_ReadMilliVolts()` reads the analog output of the module and returns the value converted to millivolts through the EduFramework Analog API [6].

`NTC_ReadMilliVolts`

Read the voltage at the analog output of the NTC module and return the value in millivolts.

Syntax

```
NTC_ReadMilliVolts(pin);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	Analog Logical Pin	Analog pin connected to the <code>AO</code> output of the NTC module.

Return value

Voltage in millivolts when successful; `-1` if the pin is invalid or the ADC measurement fails.

Example:

```
int voltageMv = NTC_ReadMilliVolts(ADC0_SE13);
```

6.2 Observing Resistance with `NTC_ReadResistance()`

`NTC_ReadResistance()` uses the measured voltage and the NTC Device voltage-divider configuration to calculate the thermistor resistance [6].

`NTC_ReadResistance`

Calculate the NTC resistance from the analog measurement and return the result in ohms.

Syntax

```
NTC_ReadResistance(pin);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	Analog Logical Pin	Analog pin connected to the <code>AO</code> output of the NTC module.

Return value

Resistance in ohms when the measurement is valid; `-1.0F` if the measurement or calculation fails.

Example:

```
float resistance = NTC_ReadResistance(ADC0_SE13);
```

Extension: Modify the program to print voltage, resistance, and temperature simultaneously in order to observe the processing sequence:

`AO voltage` → `NTC resistance` → `Temperature`.

The result can be presented in the following form:

```
Voltage: ... mV
Resistance: ... Ohm
Temperature: ... C
```

6.3 Additional NTC APIs

EduFramework also provides the following additional APIs [6]:

- `NTC_ReadRaw(pin)` returns the raw ADC value from the analog input.
- `NTC_ReadFahrenheit(pin)` returns the temperature in degrees Fahrenheit.
- `NTC_ReadThreshold(pin)` reads the digital `DO` output state of the NTC module if the hardware includes a comparator and the pin is connected.
- `NTC_SetConfig()` allows the NTC Device calculation configuration to be changed.
- `NTC_GetConfig()` retrieves the current NTC configuration.

The configuration APIs and the `DO` output are not required for the main exercise. They are useful when a different NTC type, voltage-divider configuration, or threshold-detection mode is required compared with the basic configuration [6].

Extension: Select one of the APIs above and add it to the program without changing the main temperature-reading function. For example, display the temperature in Fahrenheit or connect the `DO` pin to a Digital Input to observe the threshold state.

7 References

- [1] Massachusetts Institute of Technology, *Introduction to Electronics, Signals and Measurement*, MIT OpenCourseWare, 2006.

- [2] Gerald Recktenwald, *Temperature Measurement with a Thermistor and an Arduino* , Portland State University, 2010.
- [3] Microchip Technology Inc., *Thermistor Temperature Sensing with MCP6SX2 PGAs* , AN897, 2015.
- [4] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.
- [5] MaaZEDU, *MaaZEDU Development Board Guide* .
- [6] EduFramework, *NTC Device API* , EduFramework Source Code.